

WhisperLiveKit Web SDK 使用手册

本手册介绍 WhisperLiveKit Web SDK 的两大功能模块：**按钮转录**和**唤醒词转录**的接入与使用方法。

目录

1. [快速开始](#)
 2. [目录结构](#)
 3. [按钮转录模式 \(WhisperAsr\)](#)
 - [3.1 基本用法](#)
 - [3.1.1 使用 TranscriptRenderer 实时渲染 \(推荐\)](#)
 - [3.2 配置参数](#)
 - [3.3 方法](#)
 - [3.4 TranscriptRenderer 实时渲染器](#)
 4. [唤醒词模式 \(WhisperHotword\)](#)
 5. [回调函数参考](#)
 6. [错误处理](#)
 7. [示例代码](#)
-

1. 快速开始

1.1 环境要求

- 现代浏览器（支持 Web Audio API 和 WebSocket）

- 已运行的 WhisperLiveKit 后端服务器
- 麦克风设备（用于语音输入）

1.2 基本引入方式

在 HTML 文件中直接引入 SDK 文件：

```
<script src="WhisperAsr.js"></script>
<script src="WhisperHotword.js"></script>
<script src="Logger.js"></script>
```

2. 目录结构

```
web/sdk/
├─ Logger.js           # 日志工具类
├─ whisperAsr.js       # 按钮转录客户端
├─ whisperHotword.js   # 唤醒词转录客户端
├─ button-example.html # 按钮转录示例页面
├─ hotword-example.html # 唤醒词转录示例页面
├─ pcm_worklet.js      # PCM 音频处理工作单元
└─ recorder_worker.js  # 录音工作线程
```

3. 按钮转录模式 (WhisperAsr)

按钮转录模式需要用户主动点击"开始"按钮来启动录音，再次点击"停止"按钮结束录音。

3.1 基本用法

```
// 创建转录客户端
const client = new whisperAsr({
  serverUrl: 'ws://localhost:8000', // websocket 服务器地址
});
```

```

    language: 'zh', // 语言代码，默
    认自动检测
    autoStart: true // 连接后自动开
    始录音
  });

  // 注册转录结果回调
  client.onresult = (result) => {
    console.log('转录结果:', result.text);
  };

  // 注册实时中间结果回调
  client.onpartial = (result) => {
    console.log('中间结果:',
    result.buffer_transcription);
  };

  // 注册错误回调
  client.onerror = (error) => {
    console.error('错误:', error.message);
  };

  // 启动转录
  await client.start();

  // 停止转录
  client.stop();

```

3.1.1 使用 TranscriptRenderer 实时渲染 (推荐)

SDK 内置了 `TranscriptRenderer` 渲染器类，可以方便地实现转录结果的实时动态显示：

```

// 创建渲染器，指定显示容器
const renderer = new TranscriptRenderer({

```

```

    containerId: 'transcript', // 转录结果显示
    的容器元素ID
    className: 'transcript-line', // 转录行容器类
    名
    confirmedClass: 'confirmed', // 已确认文本的
    CSS类
    unconfirmedClass: 'buffer_transcription', // 未确认
    文本的CSS类
    waitingText: '转录结果将显示在这里...', // 等待时的提
    示文本
    autoScroll: true // 是否自动滚动
    到底部
  });

// 绑定到转录结果回调
client.onresult = (result) => {
  renderer.render(result);
};

// 转录开始前清空显示
renderer.clear();

```

HTML 容器示例:

```

<div id="transcript" class="transcript-box"></div>

<style>
.transcript-box {
  background: #f7fafc;
  border-radius: 6px;
  padding: 20px;
  min-height: 200px;
  max-height: 400px;
  overflow-y: auto;
}
.transcript-line {
  margin-bottom: 15px;
}

```

```
padding: 10px;
background: white;
border-radius: 6px;
border-left: 3px solid #667eea;
}
.transcript-line .confirmed {
  color: #333;
}
.transcript-line .buffer_transcription {
  color: #999;
  font-style: italic;
}
</style>
```

3.2 配置参数

参数	类型	默认值	说明
<code>serverUrl</code>	string	<code>ws://localhost:8000</code>	WebSocket 服务器地址
<code>language</code>	string	<code>null</code>	语言代码 (如 <code>'zh'</code> 、 <code>'en'</code>)，默认自动检测
<code>microphoneId</code>	string	<code>null</code>	麦克风设备 ID
<code>autoStart</code>	boolean	<code>true</code>	连接后是否自动开始录音
<code>logLevel</code>	string	<code>'info'</code>	日志级别: <code>debug</code> 、 <code>info</code> 、 <code>warn</code> 、 <code>error</code>
<code>logHandler</code>	Function	<code>null</code>	自定义日志处理器

3.3 方法

`whisperAsr.getMicphones()`

获取可用的麦克风设备列表。

```
const microphones = await
whisperAsr.getMicrophones();
console.log(microphones);
// 输出: [{ deviceId: 'xxx', label: '麦克风 1' },
...]
```

`client.start()`

启动转录。建立 WebSocket 连接并开始录音。

```
await client.start();
```

`client.stop()`

停止转录。停止录音并保持与服务器的连接。

```
client.stop();
```

`client.destroy()`

销毁客户端。清理所有资源并断开连接。

```
client.destroy();
```

`client.getAnalyser()`

获取音频分析器，用于可视化音频波形。

```
const analyser = client.getAnalyser();
```

3.4 TranscriptRenderer 实时渲染器

SDK 内置了 `TranscriptRenderer` 类，用于实时渲染转录结果。

3.4.1 配置参数

参数	类型	默认值	说明
<code>containerId</code>	string	必填	转录结果显示的容器元素 ID
<code>className</code>	string	<code>'transcript-line'</code>	转录行容器类名
<code>confirmedClass</code>	string	<code>'confirmed'</code>	已确认文本的 CSS 类

参数	类型	默认值	说明
<code>unconfirmedClass</code>	string	<code>'buffer_transcription'</code>	未确认文本的 CSS 类
<code>waitingText</code>	string	<code>'转录结果将显示在这里...'</code>	等待时的提示文本
<code>autoScroll</code>	boolean	<code>true</code>	是否自动滚动到底部

3.4.2 方法

方法	说明
<code>render(result)</code>	渲染转录结果, 传入 <code>onresult</code> 回调的 <code>result</code> 对象
<code>clear()</code>	清空显示内容

3.4.3 完整使用示例

```
// 1. 创建渲染器
const renderer = new TranscriptRenderer({
  containerId: 'transcript',
  confirmedClass: 'confirmed',
  unconfirmedClass: 'buffer_transcription'
});

// 2. 创建转录客户端
const client = new whisperAsr({
  serverUrl: 'ws://localhost:8000',
  language: 'zh'
});

// 3. 绑定回调
client.onresult = (result) => {
  renderer.render(result);
};

client.onpartial = (result) => {
  renderer.render(result);
};

// 4. 开始转录时清空之前的内容
renderer.clear();
await client.start();
```

4. 唤醒词模式 (WhisperHotword)

唤醒词模式持续监听麦克风，当检测到指定的唤醒词后自动开始转录。

4.1 基本用法

```
// 创建唤醒词客户端
const hotword = new whisperHotword({
  serverUrl: 'ws://localhost:8000',
  // WebSocket 服务器地址
  autoStopTimeout: 6000,
  // 用户停止说话后自动结束的时间(毫秒)
  autoStartTranscription: true
  // 检测到唤醒词后自动开始转录
});

// 注册唤醒词检测回调
hotword.onhotword = (keyword, data) => {
  console.log('检测到唤醒词:', keyword);
};

// 注册转录结果回调
hotword.onresult = (result) => {
  console.log('转录结果:', result.text);
};

// 启动唤醒词监听
await hotword.start();

// 停止唤醒词监听
hotword.stop();
```

4.2 配置参数

参数	类型	默认值	说明
<code>serverUrl</code>	string	<code>ws://localhost:8000</code>	WebSocket 服务器地址
<code>microphoneId</code>	string	<code>null</code>	麦克风设备 ID
<code>autoStopTimeout</code>	number	<code>6000</code>	用户停止说话后自动结束转录的等待时间（毫秒）
<code>autoStartTranscription</code>	boolean	<code>true</code>	检测到唤醒词后是否自动开始转录
<code>logLevel</code>	string	<code>'info'</code>	日志级别
<code>logHandler</code>	Function	<code>null</code>	自定义日志处理器

4.3 工作流程

1. 调用 `start()` 后，客户端开始监听麦克风输入
2. 当检测到唤醒词时，触发 `onhotword` 回调
3. 如果 `autoStartTranscription` 为 `true`，自动开始转录
4. 转录过程中实时触发 `onresult` 和 `onpartial` 回调
5. 用户停止说话超过 `autoStopTimeout` 毫秒后，自动结束转录
6. 转录结束后，等待 500ms，然后重新开始监听唤醒词

4.4 方法

`whisperHotword.getMicrophones()`

获取可用的麦克风设备列表（与 `WhisperAsr` 相同）。

```
const microphones = await  
whisperHotword.getMicrophones();
```

hotword.start()

启动唤醒词监听。

```
await hotword.start();
```

hotword.stop()

停止唤醒词监听。

```
hotword.stop();
```

hotword.destroy()

销毁客户端。

```
hotword.destroy();
```

5. 回调函数参考

5.1 WhisperAsr 回调

回调函数	参数	说明
<code>onresult</code>	<code>result:</code> <code>object</code>	转录结果返回时触发
<code>onpartial</code>	<code>result:</code> <code>object</code>	实时中间结果时触发
<code>onerror</code>	<code>error:</code> <code>Error</code>	发生错误时触发
<code>onconnect</code>	-	WebSocket 连接成功时触发
<code>ondisconnect</code>	-	WebSocket 断开连接时触发
<code>onready</code>	<code>config:</code> <code>object</code>	服务器配置就绪时触发
<code>onstatuschange</code>	<code>status:</code> <code>object</code>	状态变化时触发

5.2 WhisperHotword 回调

回调函数	参数	说明
<code>onhotword</code>	<code>keyword: string,</code> <code>data: Object</code>	检测到唤醒词时触发
<code>onresult</code>	<code>result: Object</code>	转录结果返回时触发
<code>onpartial</code>	<code>result: Object</code>	实时中间结果时触发
<code>onerror</code>	<code>error: Error</code>	发生错误时触发
<code>onconnect</code>	-	WebSocket 连接成功时触发
<code>ondisconnect</code>	-	WebSocket 断开连接时触发
<code>onready</code>	<code>config: Object</code>	服务器配置就绪时触发
<code>onstatuschange</code>	<code>status: Object</code>	状态变化时触发

5.3 结果对象结构

回调函数接收的 `result` 对象包含以下字段：

```
{
  status: 'active_transcription' |
  'no_audio_detected', // 转录状态
  lines: [
    // 转录行列表
    {
      speaker: 1, // 说话人 ID
```

```

        text: '转录文本', // 已确认的文本
        start: 0.0, // 开始时间（秒）
        end: 2.5, // 结束时间（秒）
        translation: null, // 翻译文本
        detected_language: 'zh' // 检测到的语言
    },
    // ... 更多行
],
    buffer_transcription: '正在识别...', // 临时缓冲的转录文本
    buffer_diarization: '', // 临时缓冲的说话人信息
    remaining_time_transcription: 0.0, // 等待转录的音频时长
    remaining_time_diarization: 0.0 // 等待说话人识别的音频时长
}

```

5.4 状态对象结构

`onstatuschange` 回调接收的状态对象：

```

{
    previous: 'idle', // 之前的状态
    current: 'ready', // 当前状态
    message: '服务器准备就绪' // 状态描述信息
}

```

5.5 状态值说明

状态值	说明
idle	初始空闲状态
connecting	正在连接服务器
connected	已连接服务器
ready	服务器准备就绪
recording	正在录音转录
listening	正在监听唤醒词
error	发生错误

6. 错误处理

6.1 错误码

错误码	说明
ALREADY_RECORDING	已经在录音中
INVALID_CONFIG	无效的配置
WEBSOCKET_ERROR	WebSocket 连接错误
MICROPHONE_ACCESS_DENIED	麦克风访问被拒绝
SERVER_ERROR	服务器错误

6.2 错误处理示例

```
client.onerror = (error) => {  
  console.error('错误代码:', error.code);  
  console.error('错误消息:', error.message);  
}
```



```
switch (error.code) {
  case 'MICROPHONE_ACCESS_DENIED':
    alert('请允许访问麦克风');
    break;
  case 'WEBSOCKET_ERROR':
    alert('无法连接到服务器，请检查服务器是否运行');
    break;
  default:
    alert('发生错误: ' + error.message);
}
};
```

7. 示例代码

7.1 按钮转录完整示例（使用 TranscriptRenderer）

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>按钮转录示例</title>
  <style>
    * { margin: 0; padding: 0; box-sizing:
border-box; }
    body { font-family: -apple-system,
BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
padding: 20px; }
    h1 { margin-bottom: 20px; }
    .btn { padding: 12px 24px; margin-right:
10px; border: none; border-radius: 6px; cursor:
pointer; font-size: 16px; }
    .btn-primary { background: #667eea; color:
white; }
```

```
.btn-primary:disabled { background: #ccc;
cursor: not-allowed; }
.btn-danger { background: #f56565; color:
white; }
.btn-danger:disabled { background: #ccc;
cursor: not-allowed; }
#status { margin: 20px 0; padding: 10px;
background: #f7fafc; border-radius: 6px; }
#transcript {
    background: #f7fafc;
    border-radius: 6px;
    padding: 20px;
    min-height: 200px;
    max-height: 400px;
    overflow-y: auto;
}
.transcript-line {
    margin-bottom: 15px;
    padding: 10px;
    background: white;
    border-radius: 6px;
    border-left: 3px solid #667eea;
}
.transcript-line .confirmed { color: #333; }
.transcript-line .buffer_transcription {
color: #999; font-style: italic; }
</style>
</head>
<body>
    <h1>按钮转录演示</h1>

    <button id="startBtn" class="btn btn-primary">开
始录音</button>
    <button id="stopBtn" class="btn btn-danger"
disabled>停止录音</button>

    <div id="status">状态： 空闲</div>
```

```
<div id="transcript">
  <p style="color: #999; text-align: center;">
转录结果将显示在这里...</p>
</div>

<script src="Logger.js"></script>
<script src="whisperAsr.js"></script>
<script>
  // 创建 TranscriptRenderer 渲染器
  const renderer = new TranscriptRenderer({
    containerId: 'transcript',
    confirmedClass: 'confirmed',
    unconfirmedClass: 'buffer_transcription'
  });

  // 创建转录客户端
  const client = new whisperAsr({
    serverUrl: 'ws://localhost:8000',
    language: 'zh'
  });

  // 状态变化回调
  client.onstatuschange = (status) => {

    document.getElementById('status').textContent =
      `状态: ${status.current} -
${status.message}`;

    document.getElementById('startBtn').disabled =
status.current !== 'idle';

    document.getElementById('stopBtn').disabled =
status.current !== 'recording';

    // 开始新转录时清空显示
    if (status.current === 'ready') {
```

```

        renderer.clear();
    }
};

// 转录结果回调 - 使用渲染器
client.onresult = (result) => {
    renderer.render(result);
};

// 中间结果回调 - 使用渲染器实现实时显示
client.onpartial = (result) => {
    renderer.render(result);
};

// 错误回调
client.onerror = (error) => {
    alert('错误: ' + error.message);
};

// 绑定按钮事件
document.getElementById('startBtn').onclick
= async () => {
    try {
        await client.start();
    } catch (e) {
        console.error(e);
    }
};

document.getElementById('stopBtn').onclick =
() => {
    client.stop();
};
</script>
</body>
</html>

```

7.2 唤醒词转录完整示例（使用 TranscriptRenderer）

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>唤醒词转录示例</title>
  <style>
    * { margin: 0; padding: 0; box-sizing:
border-box; }
    body { font-family: -apple-system,
BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
padding: 20px; }
    h1 { margin-bottom: 20px; }
    .btn { padding: 12px 24px; margin-right:
10px; border: none; border-radius: 6px; cursor:
pointer; font-size: 16px; }
    .btn-primary { background: #f5576c; color:
white; }
    .btn-primary:disabled { background: #ccc;
cursor: not-allowed; }
    .btn-danger { background: #4a5568; color:
white; }
    .btn-danger:disabled { background: #ccc;
cursor: not-allowed; }
    #status, #hotword { margin: 20px 0; padding:
10px; background: #fff5f5; border-radius: 6px; }
    #hotword { background: #f0fff4; }
    #transcript {
      background: #fff5f5;
      border-radius: 6px;
      padding: 20px;
      min-height: 200px;
      max-height: 400px;
      overflow-y: auto;
```

```

    }
    .transcript-line {
        margin-bottom: 15px;
        padding: 10px;
        background: white;
        border-radius: 6px;
        border-left: 3px solid #f5576c;
    }
    .transcript-line .confirmed { color: #333; }
    .transcript-line .buffer_transcription {
color: #999; font-style: italic; }
</style>
</head>
<body>
    <h1>唤醒词转录演示</h1>

    <button id="startBtn" class="btn btn-primary">开
始监听</button>
    <button id="stopBtn" class="btn btn-danger"
disabled>停止监听</button>

    <div id="status">状态： 空闲</div>
    <div id="hotword">唤醒词： 等待检测...</div>
    <div id="transcript">
        <p style="color: #999; text-align: center;">
转录结果将显示在这里...</p>
    </div>

    <script src="Logger.js"></script>
    <script src="WhisperAsr.js"></script>
    <script src="WhisperHotword.js"></script>
    <script>
        // 创建 TranscriptRenderer 渲染器
        const renderer = new TranscriptRenderer({
            containerId: 'transcript',
            confirmedClass: 'confirmed',
            unconfirmedClass: 'buffer_transcription'

```

```

});

// 创建唤醒词客户端
const hotword = new whisperHotword({
  serverUrl: 'ws://localhost:8000',
  autoStopTimeout: 6000
});

// 状态变化回调
hotword.onstatuschange = (status) => {

  document.getElementById('status').textContent =
    `状态: ${status.current} -
    ${status.message}`;

  document.getElementById('startBtn').disabled =
    status.current !== 'idle';

  document.getElementById('stopBtn').disabled =
    status.current === 'idle';

    // 重新开始监听时清空显示
    if (status.current === 'listening') {
      renderer.clear();
    }
};

// 唤醒词检测回调
hotword.onhotword = (keyword, data) => {

  document.getElementById('hotword').textContent =
    `唤醒词: ${keyword}`;

};

// 转录结果回调 - 使用渲染器
hotword.onresult = (result) => {

```

```
        renderer.render(result);
    };

    // 中间结果回调 - 使用渲染器实现实时显示
    hotword.onpartial = (result) => {
        renderer.render(result);
    };

    // 错误回调
    hotword.onerror = (error) => {
        alert('错误: ' + error.message);
    };

    // 绑定按钮事件
    document.getElementById('startBtn').onclick
= async () => {
        try {
            await hotword.start();
        } catch (e) {
            console.error(e);
        }
    };

    document.getElementById('stopBtn').onclick =
() => {
        hotword.stop();
    };
</script>
</body>
</html>
```

附录

A. 浏览器兼容性

浏览器	最低版本
Chrome	56+
Firefox	52+
Safari	14.1+
Edge	79+